

Tabla comparativa de desempeño:

Modelo	Función de Activación	Función de Pérdida	Algoritmo de optimización	Actualizaciones por época	Learning Rate	Accuracy	Comportamiento
Neurona 1	Sigmoide	Binary Cross-Entropy	SGD + Momentum	70	Fijo	1.0	Descenso gradual y constante durante las 1,000 épocas.
Neurona 2	Sigmoide	Binary Cross-Entropy	Batch + Adam	1	Fijo con valor base. Su tamaño de actualización es adaptativo por parámetro.	1.0	Caída casi instantánea a valores cercanos a 0 desde el inicio del entrenamiento logrando una convergencia acelerada.

Observación:

Momentum: procesa muestra por muestra de forma aleatoria (SGD estocástico puro) dentro de un bucle interno. Se calcula el gradiente individual por muestra.

Adam: utiliza todo el lote de entrenamiento por época (Batch completo). Se calcula el gradiente global por época.

Conclusión:

Se puede apreciar que todo mi código es exactamente igual a excepción del algoritmo de optimización. En ambos casos se utiliza una neurona simple con una función de activación sigmoide. En mi código opté por clasificar solamente dos clases del conjunto de datos Iris y, con los resultados obtenidos, es observable que ambos modelos son capaces de hacer una clasificación certera. Mi función de activación sigmoide transforma los valores de los resultados en un valor entre 0 y 1, con un umbral de 0.5 para poder clasificar dichos valores en sus respectivas clases.

La principal diferencia en mis dos modelos es la forma en la que se actualizan los pesos y sesgos. SGD con Momentum utiliza el gradiente descendiente calculado a partir de una muestra individual y mantiene una memoria de las actualizaciones anteriores mediante el término de Momentum. Esto permite que la dirección de actualización considere información de los gradientes anteriores. En el caso de Batch con Adam, se mantienen estimaciones del primer y segundo momento del gradiente. Adicionalmente, adapta la magnitud de la actualización de cada parámetro, utilizando una tasa de aprendizaje diferente de acuerdo con el comportamiento de los gradientes.

A mi parecer, las diferencias en los resultados en la gráfica de Pérdida vs Épocas pueden atribuirse a las diferencias en la tasa de aprendizaje variable (en el caso de Adam) y en el algoritmo de optimización utilizado, porque todos los factores son los mismos: épocas, división de datos de entrenamiento y prueba y normalización de datos. El hecho de que un algoritmo utilice SGD y otro Batch explica el motivo por el cual Adam avanza de forma directa al mínimo, mientras que Momentum tiene una dinámica más lenta. Ambos modelos son válidos, pero su actualización de pesos y sesgos hacen que la función de pérdida alcance el mínimo con un número diferente de épocas.

Los resultados demuestran que Adam superó en velocidad de convergencia a SGD con Momentum en este escenario en específico. Esto se debe a que Adam ajusta la tasa de aprendizaje individualmente para cada parámetro utilizando estimaciones de los momentos primero y segundo, lo que permite minimizar el error de forma casi inmediata. Como Adam utiliza Batch, permite calcular un gradiente más estable por época, mientras que SGD con Momentum procesa el gradiente muestra por muestra. Por ende, Momentum introduce una dinámica estocástica que suaviza las actualizaciones, pero requiere una trayectoria de descenso prolongada.

Este ejercicio permitió comprobar de manera pragmática la manera en que diferentes algoritmos de optimización pueden modificar el proceso de aprendizaje de una misma neurona, aunque se utilice la misma función de activación. En ambos casos, se puede reflejar lo aprendido en clase, el Momentum acumula inercia para evitar oscilaciones y Adam combina las ventajas de los momentos con tasas adaptativas para maximizar la estabilidad del gradiente. Los resultados también permiten relacionar los conceptos matemáticos del gradiente, Momentum, estimación de momentos con su implementación práctica en Python.